Arrays, 2D & 3D, and ArrayLists

Intermediate Object-Oriented Programming · Java

Core Java, Vol. I & II (13th ed.)

WHERE WE'RE GOING

The arc

1

Get our bearings

1-D array review, then 2-D arrays — how Java really models a grid.

2

More dimensions

Ragged & 3-D arrays, then the Arrays class that does the busywork.

3

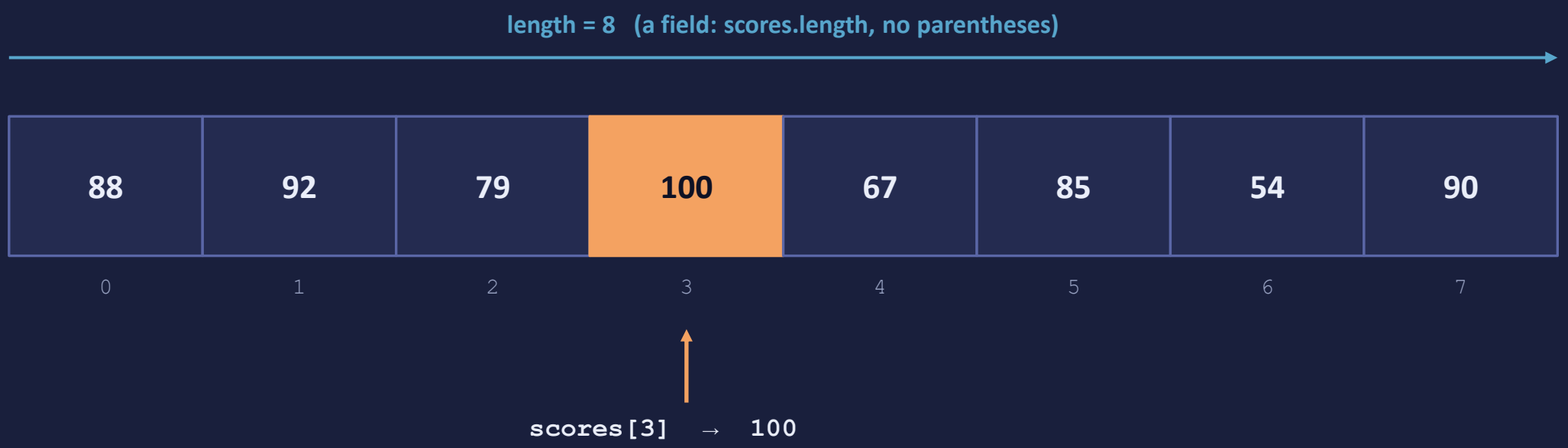
Get flexible

ArrayList: grow, shrink, iterate — and the handful of traps that bite once.

The slides give you the picture; class time is for predicting what code prints, then running it to find out.

REVIEW · 1-D ARRAYS

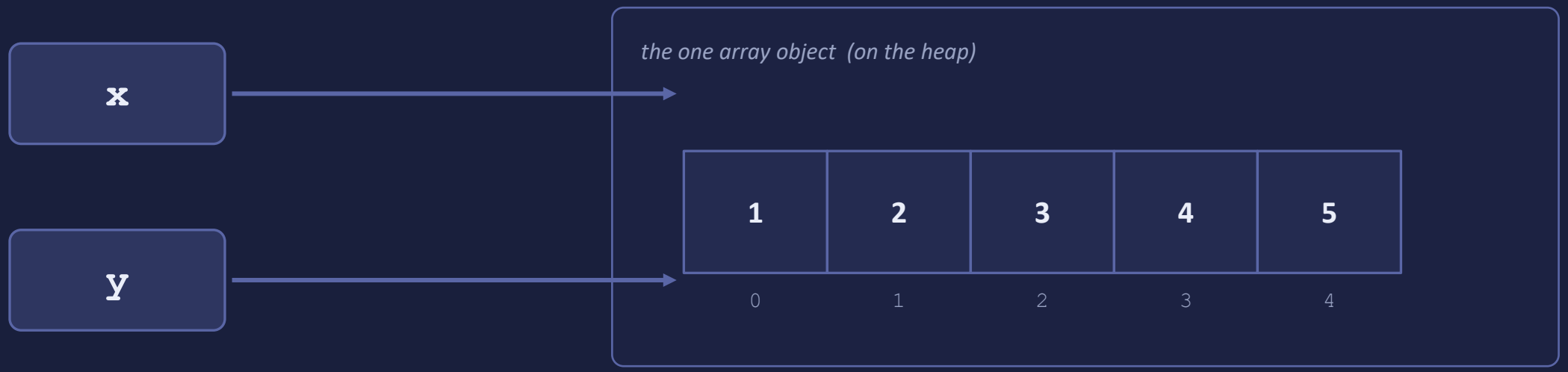
An array is a row of indexed boxes




```
int[] scores = new int[8];   // 8 slots, each defaults to 0
```

Indices run 0 … length-1. The last valid index is length-1 — never length.

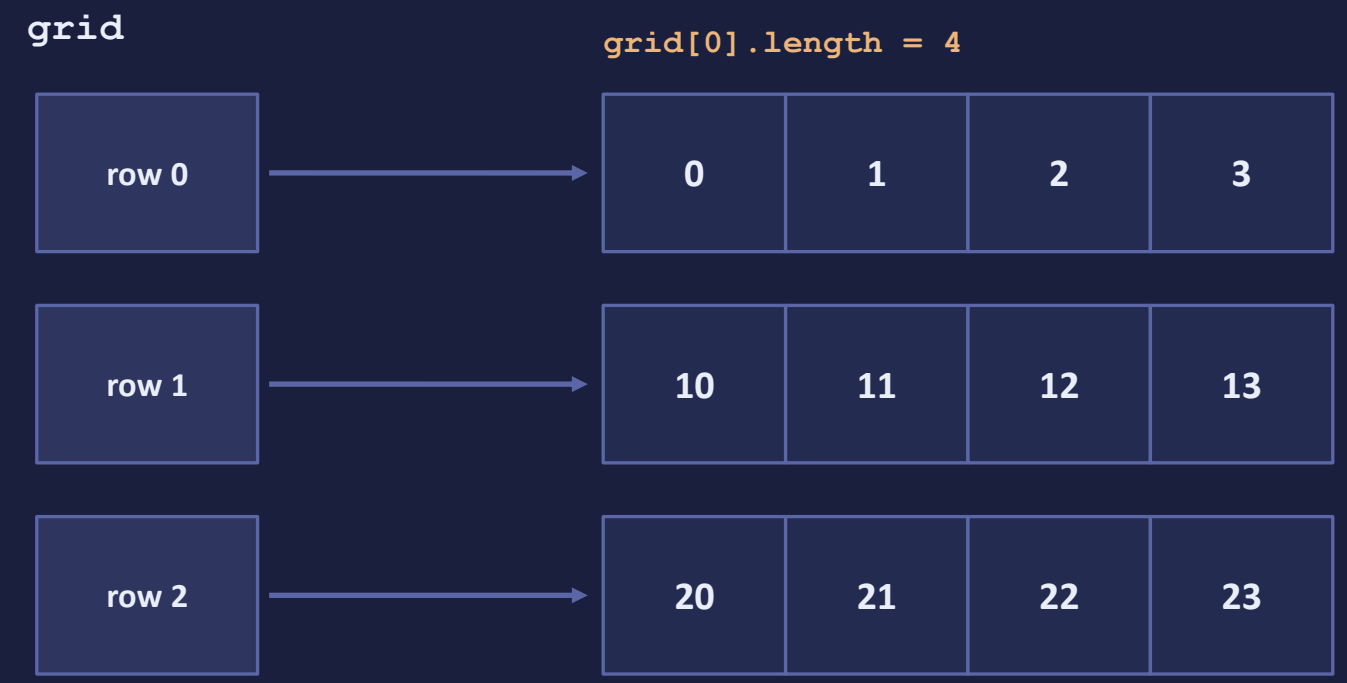
The variable is a label, not the box




```
int[] y = x;   // y and x now point at the SAME array
```

Change through y and x sees it too. Want an independent copy? Arrays.copyOf(x, x.length).

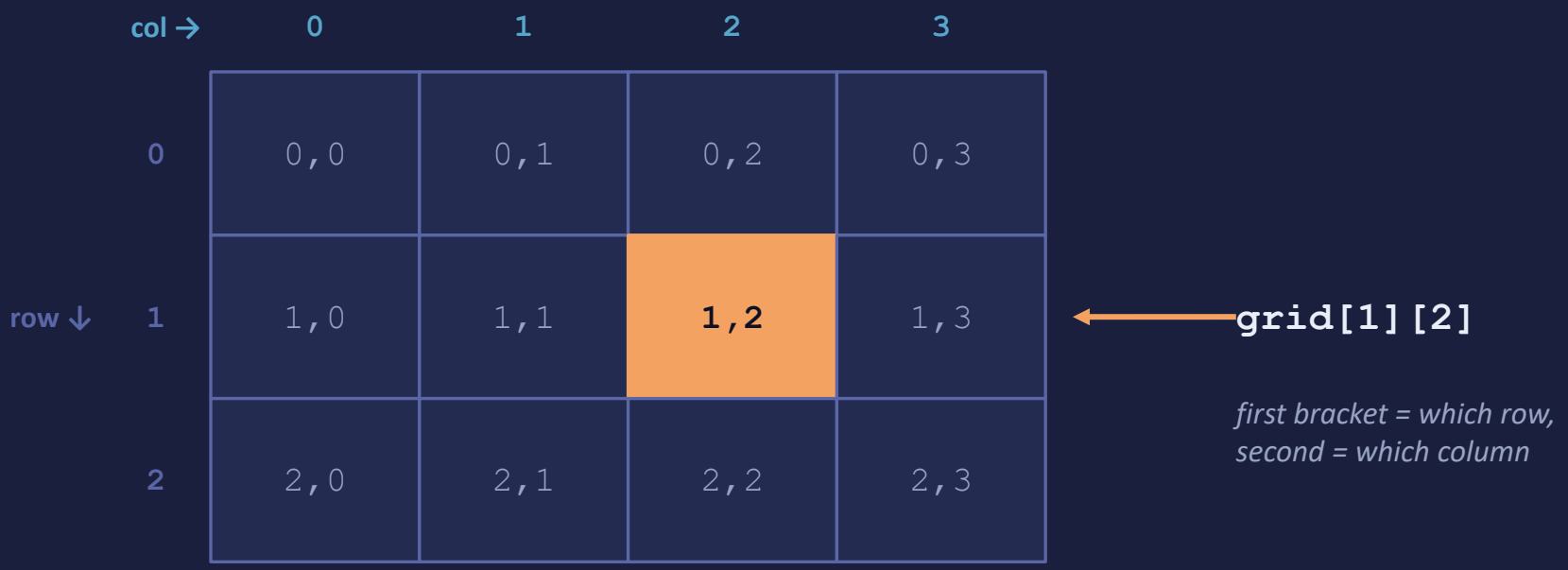
A 2-D array is an array of arrays




```
int[][] grid = new int[3][4];   // 3 rows, each its own int[4]
```

Internalize this: the outer array holds references to rows. Everything else follows.

Indexing: grid[row][col]



grid.length is the number of rows; grid[r].length is that row's width.

Rows need not be the same length

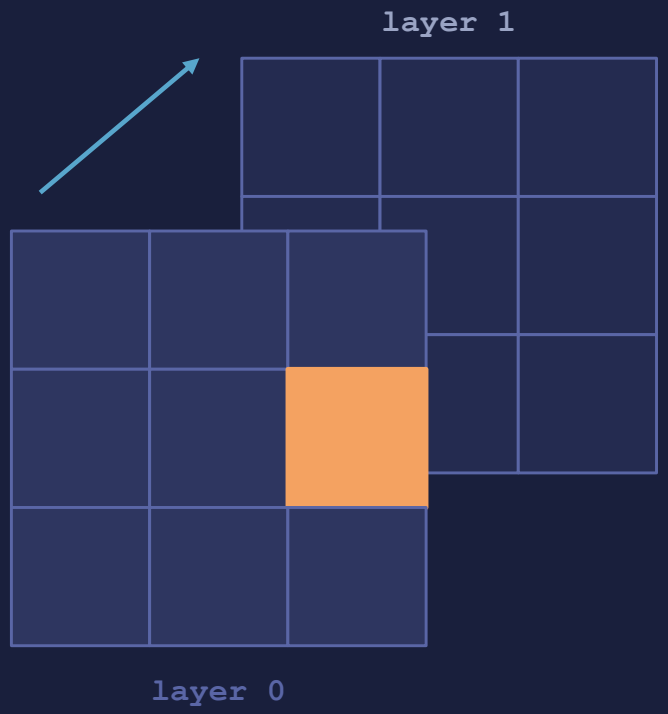

```
int[][] tri = new int[4][];   // 4 row
slots — each row still null
```

Then allocate each row on its own:
`tri[r] = new int[r + 1];`

Natural fit for a triangle, a lower-triangular matrix, or "games each player played this week."

Right after new int[4][], every row is null — the outer array exists, the rows don't yet.

Stack grids into layers



Read the indices outside-in:

`cube[ layer ]`
which grid

`cube[ layer ][ row ]`
which row in it

`cube[ layer ][ row ][ col ]`
which column

```
int[][][] cube = new int[2][3][3];
```

Rare but real: voxel worlds, stacks of image frames, values sampled over time.

TOOLKIT

The Arrays class does the busywork

`Arrays.sort(a)`

orders it in place, ascending

`Arrays.fill(a, v)`

sets every slot to v

`Arrays.copyOf(a, n)`

"resize" — new array, copied over

`Arrays.equals(a, b)`

same contents? (not same object)

`Arrays.binarySearch(a, k)`

fast lookup — sorted arrays only

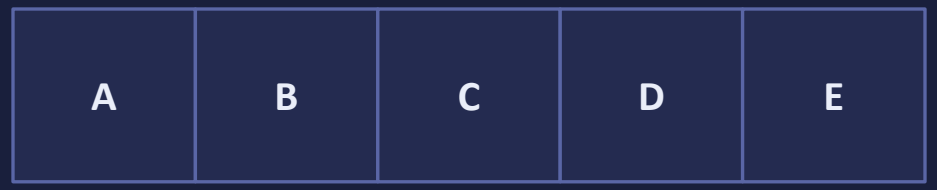
`Arrays.deepToString(a)`

readable printing of nested arrays

Stop hand-coding this. Learn these six and you'll rarely write array bookkeeping again.

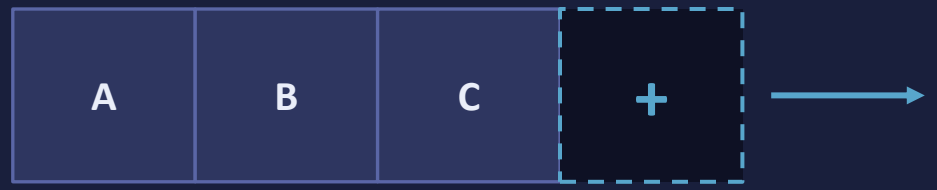
When the count won't hold still

`array`



length frozen at creation — no more, no fewer

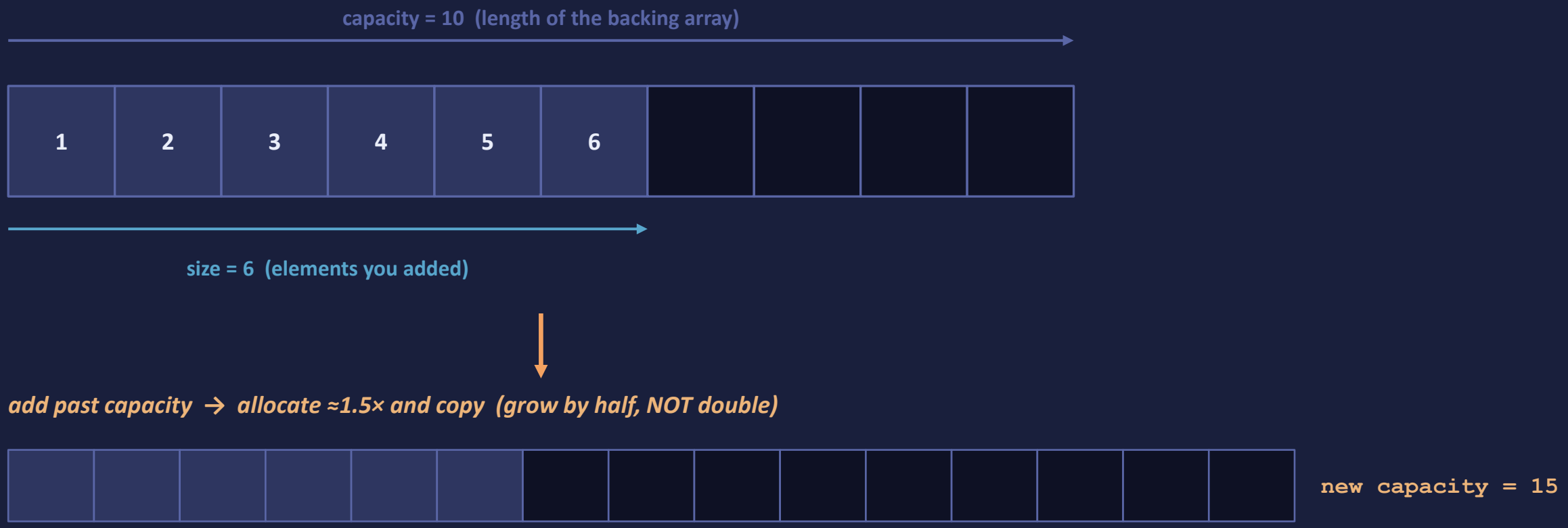
`ArrayList`



add() when you need more; remove() when you don't

An ArrayList wraps an array and, when it fills up, quietly copies into a bigger one for you.

Size is what you added; capacity is the room

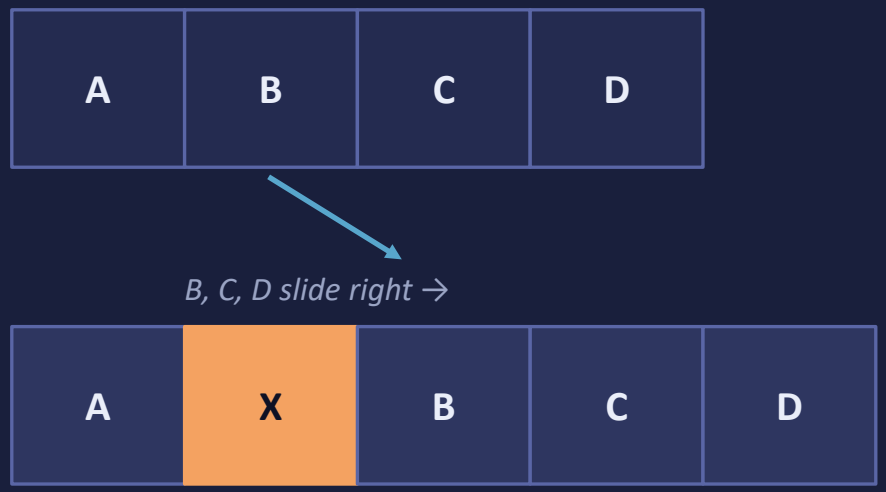


Default capacity 10, allocated on the first add. Know the count up front? new ArrayList<>(n) skips the copies.

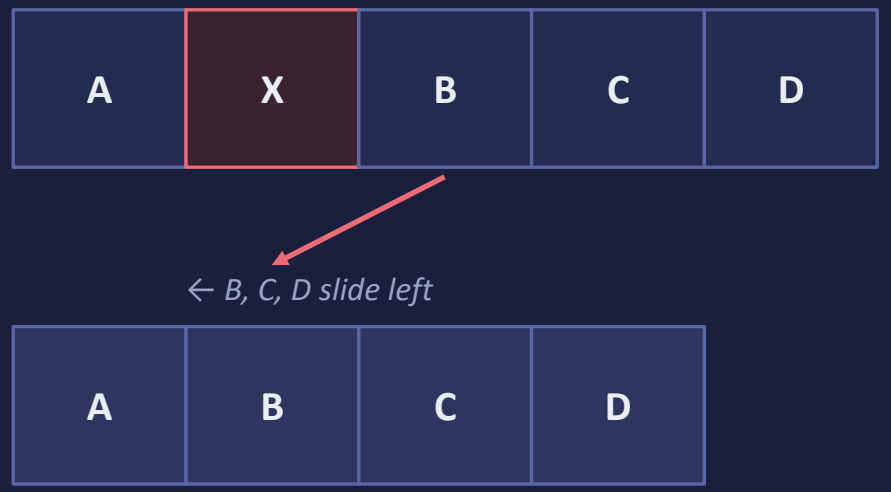
ARRAYLIST · OPERATIONS

Insert and remove shift the neighbors

`list.add(1, "X")`



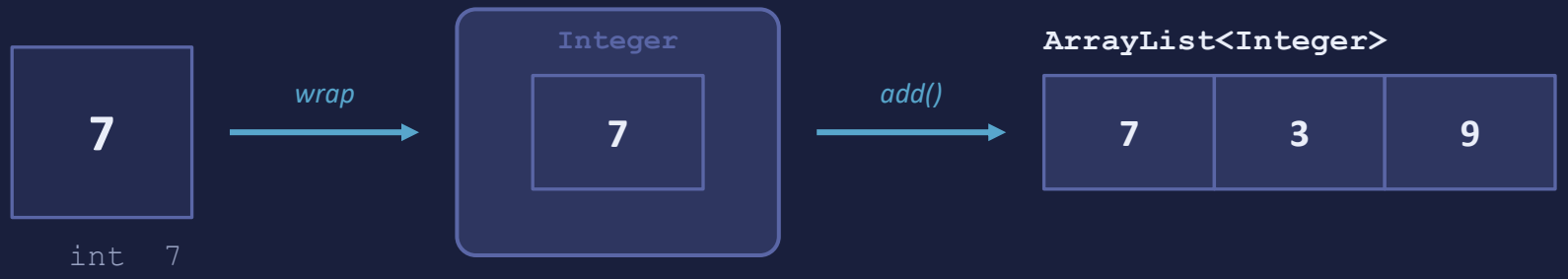
`list.remove(1)`



Appending at the end is cheap. Inserting or removing in the middle moves everything after it.

ARRAYLIST · GENERICS

There is no ArrayList<int> — Java boxes for you



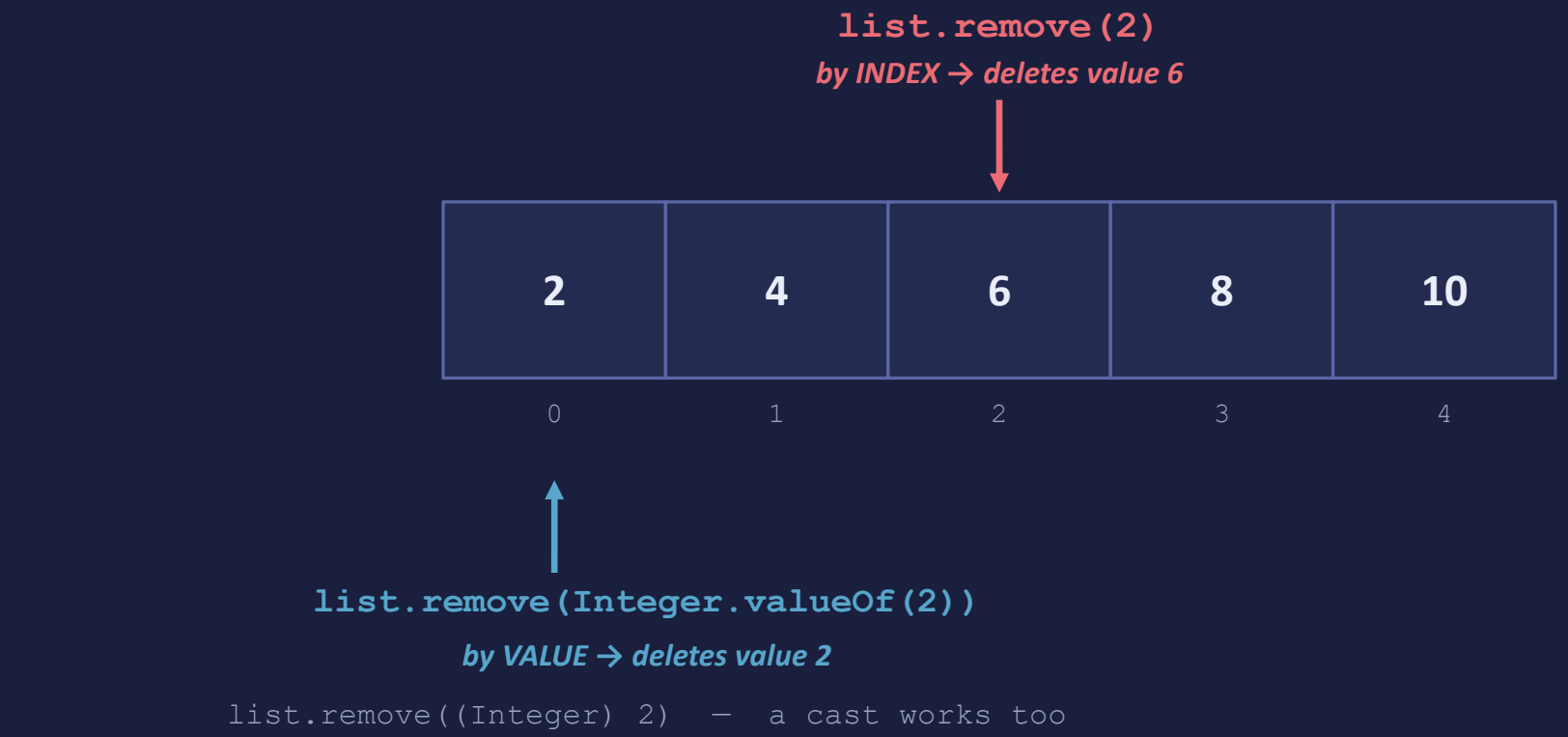

```
ArrayList<Integer> nums = new ArrayList<>();  nums.add(7);  int x = nums.get(0);
```

Numbers auto-wrap going in and auto-unwrap coming out. Convenient — but each becomes a heap object.

Millions of numbers in a hot loop? A plain int[] is the honest choice.

GOTCHA

remove(2) is not remove(Integer.valueOf(2))



`list.remove((Integer) 2)` — a cast works too

On an ArrayList<Integer>, an int argument calls remove(int index); an Integer calls remove(Object). Memorize this one cold.

DECIDE

Array or ArrayList?

Reach for an array

- size is fixed and known
- primitives in a performance-sensitive spot
- grids, boards, matrices (multi-dimensional)
- an API hands you or wants arrays

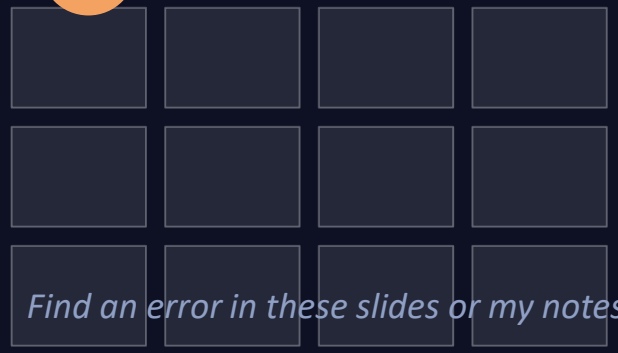
Reach for an ArrayList

- the count grows or shrinks at runtime
- you don't know the count up front
- you want contains / indexOf / removeIf
- the everyday "a bunch of things" need

Rule of thumb: fixed grid of primitives → array. Growing pile of objects → ArrayList.

Your turn

1. **Read** the arrays & ArrayList material in the book — and my notes
2. **Practice** run every example, then break it and see what happens
3. **Demo** build a project and show it to me working



Find an error in these slides or my notes? Tell me — that's the assignment behind the assignment.